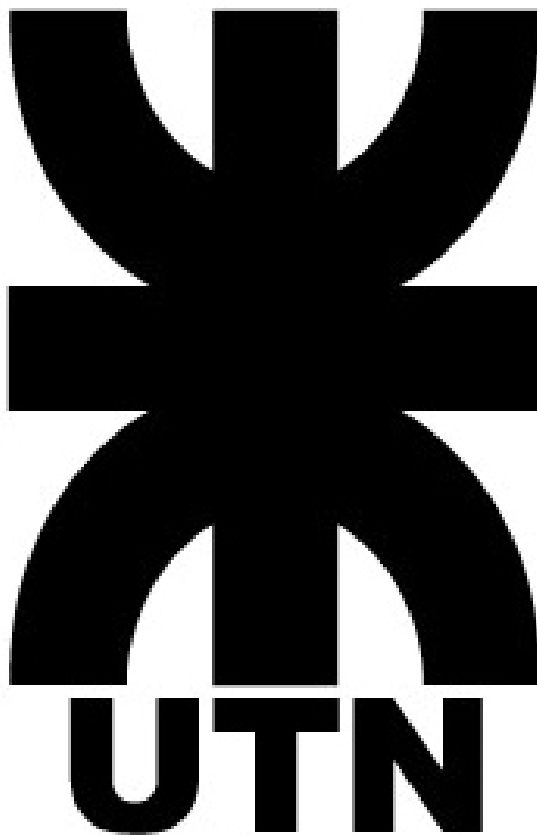


# Proyecto Final - Problema

Esteban Bustamante - UTN Facultad Regional La Rioja  
Profesor: Ing. Oscar Almonacid

2024



# Índice

|                          |          |
|--------------------------|----------|
| <b>Índice</b>            | <b>2</b> |
| <b>1. Introducción</b>   | <b>3</b> |
| <b>2. Problema</b>       | <b>3</b> |
| 2.1. ¿Qué? . . . . .     | 3        |
| 2.2. ¿Por qué? . . . . . | 4        |
| 2.3. ¿Dónde? . . . . .   | 4        |
| 2.4. ¿Cuándo? . . . . .  | 4        |
| 2.5. ¿Quién? . . . . .   | 4        |
| 2.6. ¿Cómo? . . . . .    | 4        |
| <b>3. Bibliografía</b>   | <b>6</b> |
| <b>Índice de figuras</b> | <b>6</b> |

## 1. Introducción

Tal como fue sugerido por la cátedra, se utilizará la metodología  $5W + 1H$  para el desarrollo del problema a ser resuelto en el trabajo final de la carrera. Debe ser dicho que si bien hay un enfoque tanto innovador como pedagógico en el mismo, por sobre todo se busca explotar la capacidad de diseño de un sistema, de forma tal que se logre una solución realmente adecuada y eficiente. Es decir que no solo se buscará aprender y enseñar de forma anecdótica, sino realmente buscar la mejor opción para la solución del problema planteado, de igual forma que lo realizaría una empresa.

## 2. Problema

### 2.1. ¿Qué?

El problema que se presenta en cuestión es el desarrollo de un firmware para un Decodificador MPEG-2 VLSI prototipado en FPGA, conforme con la norma ISO/IEC 13818-4, con un submuestreo 4:2:0. En la Figura 1 observamos el bloque completo. Es importante notar que la entrada es un bitstream en formato MPEG-2 y la salida es RGB, es decir que podríamos conectar nuestra salida tranquilamente a un HDMI y visualizar un video en un alguna pantalla.

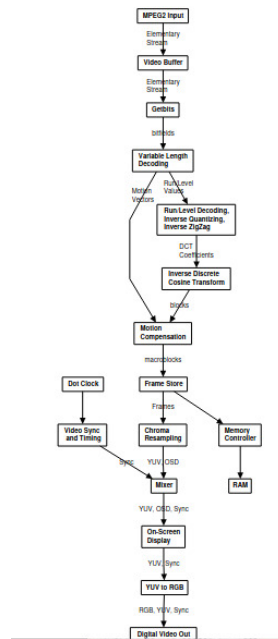


Figura 1: Pipeline del Decoder MPEG-2

## 2.2. ¿Por qué?

El decodificador en sí no lleva en su arquitectura lazos de control ni tampoco tiene un microprocesador embebido que lo configure para su funcionamiento. Resulta crítico el desarrollo de un firmware robusto que permita controlar en tiempo real el bitstream del decodificador, y por supuesto que se encargue de la inicialización correcta del mismo, ya que en cualquier sistema que se piense para su integración el tráfico de bits no debe cortarse o en todo caso si se interrumpe su normal funcionamiento debiera claramente saberse el causante del error.

## 2.3. ¿Dónde?

En todos los sistemas donde se utilizan aceleradores por hardware de sistemas de visión artificial, especialmente en este caso VLSI aplicable a dispositivos de bajo consumo de potencia como aquellos que funcionan a batería. Por ej, podríamos tener un Codec MPEG-2 por hardware en un Smartphone que pretenda tener reproducción de videos a una alta tasa de cuadros por segundo.

## 2.4. ¿Cuándo?

Históricamente, en las primeras épocas los gráficos fueron una cuestión manejada principalmente por software, con sistemas muy básicos con capacidades muy limitadas destinadas a mostrar pixeles en los monitores de las primeras computadoras. A medida que pasaron los años, comenzó la transición con la aparición de las GPU (*Graphical Processing Unit*), que optaron por un punto medio, donde si bien el hardware ya era mucho más específico, los algoritmos en sí seguían corriendo todos por software. Hasta llegar a nuestros días, donde ciertas aplicaciones críticas en velocidad y en consumo de potencia como los mencionados en 2.3 precisan que algoritmos y ciertos tipos de codificaciones corran por hardware.

## 2.5. ¿Quién?

Un ejemplo de empresa que podría requerir un controlador de gráficos como este sería el caso de la organización openMSP430, que sería la versión open source de la familia de microcontroladores de Texas Instruments. En este caso, ya está hecho el openGFX430, que consiste en un controlador que entre sus bloques posee un decodificador MPEG-2. En nuestro caso, recordemos que el bloque es provisto por la página <https://opencores.org/>.

## 2.6. ¿Cómo?

Para la solución de este problema se propone hacer uso de una placa de desarrollo en FPGA (*Microchip® PolarFire Discovery Kit*) para llevar a cabo el prototipado del sistema y con su correspondiente firmware. La idea sería sintetizar el diseño en la placa propuesta, y desarrollar el firmware en los microprocesadores propios del SoC de la placa, que consiste en un subsistema de cores RISC-V, algo cada vez más común en la industria de los semiconductores. Resultaría propicio este modo debido a la flexibilidad propia de esta arquitectura, siempre pensando en que este sistema comercialmente sería un ASIC embebido

en un controlador de gráficos, por lo cual esta placa nos permitiría tranquilamente emular las frecuencias de trabajo y el entorno propio de un chip VLSI.

El firmware consistiría en general un sistema de registros de configuración, una secuencia de inicialización, y programar lazos de control sobre el sistema en tiempo real. Opcionalmente podría agregarse en hardware un árbitro para generar un estándar de comunicación propio de un ASIC, como podría ser en AXI4 o AHB.



### 3. Bibliografía

- De Vleeschauwer, Koen. *MPEG-2 Decoder User Guide*. 2017. <https://opencores.org/websvn/filedetails?repname=mpeg2fpga&path=%2Fmpeg2fpga%2Ftrunk%2Fdoc%2Fmpeg2fpga.pdf>

### Índice de figuras

1. Pipeline del Decoder MPEG-2 . . . . . 3